



**Università
di Genova**

DIBRIS DIPARTIMENTO
DI INFORMATICA, BIOINGEGNERIA,
ROBOTICA E INGEGNERIA DEI SISTEMI

Template for Master's Theses at DIBRIS

Michele Frattini

Master's Thesis

Università di Genova, DIBRIS
Via Dodecaneso, 35 16146 Genova, Italy
<https://www.dibris.unige.it/>



Computer Science MSc
Software Security and Engineering Curriculum

Template for Master's Theses at DIBRIS

Michele Frattini

Advisor: Advisor Name

Examiner: Examiner Name

November, 2025

Abstract

This is a template for the Master's thesis in Computer Science at the University of Genoa.

This document doubles as a guide for how to structure and write it. Please keep a copy of this advice and read it carefully while you write your thesis by replacing it with your material.

Table of Contents

Chapter 1	Introduction	5
1.1	Style and Language	5
1.2	Motivation	7
1.3	Content of the Thesis	7
1.4	L ^A T _E X Tricks	7
Chapter 2	Related Work	10
2.1	Sources	11
2.2	Style	12
Chapter 3	Background	14
Chapter 4	Methodology	16
Chapter 5	Results	18
5.1	Discussion	19
Chapter 6	Conclusion	20

Chapter 1

Introduction

This is a template for the Computer Science Master’s Theses at *dipartimento di informatica, bioingegneria, robotica e ingegneria dei sistemi* (DIBRIS), University of Genoa.¹ It uses the L^AT_EX class `masterthesis.cls` by Davide Ancona. We will use this template to discuss what is generally expected in the structure of a Master’s Thesis. While reading academic documents, you will find that they generally follow a similar structure.

This structure is generally good for research documents, where your main goal is to do something new. In other cases (e.g., the goal of your thesis is to review a topic and provide an understanding of the field), a different structure may be more desirable. In any case, discuss the structure of your thesis with your advisor.

I have put hints and links through the document for tools you can use; some may become stale over time, and something better may appear. If you find some that are not covered here, consider sending me an email or a pull request to integrate them!

1.1 Style and Language

We are writing a scientific document, and we should consider that skepticism is at the core of the scientific method. We should be skeptical of our work, and we can see the thesis as an effort to persuade a skeptical reader that we have done a piece of work that is both useful and solid. Whenever you state something, you should give proof: either by referencing one or more authoritative documents or proving it yourself. Avoid marketing-speak (“our work is poised to revolutionize...”).²

¹<https://dibris.unige.it>

²Unless you can prove what you say. But these statements are mostly impossible to prove... By the way, notice that footnotes should be put after punctuation as in this case.

Do not try to reach a given number of pages. Instead, try to make it *complete*, *clear* and *concise*. Try to give all the information that will convince the reader that your work is solid and that you have done a good job; use simple language and consistent terminology. If you can say something more simply, do it. Avoid weasel words³ and, when you can, the passive voice. “We consider this to be not a security risk” is much clearer than “This is not considered a security risk” because it makes it explicit who makes the claim, and takes the responsibility for it.

Consider that your reader will be a fellow computer scientist, but not necessarily an expert on your topic. Try to give them the information they need to understand your work, but do not lose your reader’s time and attention with unnecessary details.

Documents like this generally follow a structure like this one. It is not a strict requirement, but if you follow it readers will likely find it easier to navigate. Consider that in scientific documents some readers will not read the whole document, but will look for specific pieces of information. A clear structure will help them.

Chapters generally start with a short introduction to their topic. This is followed by the main content of the chapter, and then by a conclusion that summarizes the main points. Note that both chapter and section titles are capitalized.⁴

Introduce the expanded form of acronyms the first time you use them, excepting only those so common that you can reasonably take for granted for any reader (e.g., CPU or PDF).

For this document, you should use formal writing. Do not use contractions⁵ such as *don’t* or *it’s*.

Be extremely wary of large language models (LLMs) like Gemini, ChatGPT, or Copilot.⁶ You take responsibility for the content of your thesis and declare you have written it yourself. If you want a tool that helps you to write in better English, consider using something like Grammarly⁷; it is also supported by the Overleaf online L^AT_EX editor.⁸ Note that Overleaf now has a generative AI option: do not use it without declaring it, as it would violate its terms of service⁹ which, among other things, forbids “to mislead any third party that any output from Grammarly’s generative AI was solely human generated”.

³https://en.wikipedia.org/wiki/Weasel_word

⁴<https://www.grammarly.com/blog/capitalization-in-the-titles/>

⁵<https://www.grammarly.com/blog/contractions/>

⁶Note that we used the serial comma (https://en.wikipedia.org/wiki/Serial_comma) here. It is a good idea to use it as it can remove ambiguity from your text.

⁷<https://grammarly.com>

⁸https://www.overleaf.com/learn/how-to/Use_Grammarly_with_Overleaf

⁹<https://www.grammarly.com/terms>

1.2 Motivation

This is the first important thing you should write in your Introduction. You can see it as the effort to convince your reader that the problem you are addressing *matters*. Mention that the problem is unsolved, but you will discuss that at you’re facing a skeptical reader asking “is this useful?”.

Sometimes it will be obvious why the problem matters, but in other cases, this will be one of the most delicate parts of your work. Of course, your advisor will help you with this.

1.3 Content of the Thesis

The other main goal of the Introduction is to give the reader an overview of what they will find in the document. There is no concept of “spoiler” in scientific documents: summarize what the reader will find in every chapter. Avoid obvious statements that help nobody such as, for example, “*In the Related Work chapter we discuss relevant pieces of work in the state of the art*”; provide synthetic but useful information instead (e.g., “*In Chapter 2 we discuss other density-based clustering algorithms and highlight the differences to our approach*”). This will help them to navigate it (and to find what they’re interested in, if they are reading the whole document).

I am not a fan of those “*This document is structured as follows*” paragraphs. My advice is to expose the content of the thesis and give references, organically, to the parts of it that treat them while you are at it.

The Introduction is arguably the most important chapter of an academic text because, after reading it, one should understand the key concepts of your work. The rest of the document “only” explains the details; you can expect somebody to stop at the Introduction alone.¹⁰

1.4 L^AT_EX Tricks

If you’re using L^AT_EX to write your document, you can use some tricks to make your life easier. This document uses the `\Cref`¹¹ command to conveniently reference sections, figures, and tables, and `\textcite` and `\authorcite` to cite paper authors. You can also

¹⁰Likewise, while you are going through somebody else’s work, it makes sense to stop at the Introduction (and maybe the Conclusion) if you’re interested only in a high-level overview.

¹¹I also created a `\fullref` to generate text like “**Introduction** (Chapter 1)”. Check the source if you are interested.

use `\xspace` to define your macros and avoid spacing problems. Check the source code of this document to see how they are used.

`rubber` is a good tool to compile $\text{\LaTeX}$ documents. You can run the command `rubber -d main` to compile this document to PDF, while running `pdflatex` and `biber` (or `bibtex`) the right number of times.

The Overleaf¹² online editor is a good tool to synchronize with your advisor; it exports a git repository so you can edit your work online and offline.

To handle acronyms, the `acronym`¹³ package is a good resource to automatically introduce an acronym's expanded form the first time you use it. It is also possible to create an acronyms table; consider using it if you use many acronyms in your paper and you think it may help the reader. Once again, you can have a look at how it's used in the source of this document.

You are likely going to cite several websites in your bibliography. Here is documentation¹⁴ on how to do that in both BibTeX and BibLaTeX. You may want to use a reference management software like Zotero¹⁵ or Mendeley¹⁶ to handle your bibliography database.

Your supervisors and reviewers will likely go over your document in several revisions; I'm sure they would really be thankful to you if you used a way to clearly show what changed since their previous pass. `Latexdiff`¹⁷ is a tool to do just that; you can also use it on Overleaf¹⁸. Another possibility is tracking the changes on the PDF output, using tools like Draftable¹⁹.

Advisors may have very different preferences in how to share feedback. A possibility is the `todonotes`²⁰ package. This document's `preamble.tex` source file shows an example of using it to include comments; note that they can be removed from the PDF output by passing the `final` option to the document class (i.e., `\documentclass[final]{masterthesis}` in the main `tex` file.

MD: This is an example of a note using `todonotes`.

IMPORTANT: Make sure the curriculum on the cover pages and the department address matches your own. Currently, you'll have to edit the `masterthesis.cls`

¹²<https://www.overleaf.com>

¹³<https://ctan.org/pkg/acronym>

¹⁴<https://bibtex.eu/faq/how-can-i-use-bibtex-to-cite-a-website/>

¹⁵<https://www.zotero.org/>

¹⁶<https://www.mendeley.com/>

¹⁷<https://ctan.org/pkg/latexdiff>

¹⁸https://www.overleaf.com/learn/latex/Articles/How_to_use_latexdiff_on_Overleaf

¹⁹<https://www.draftable.com/>

²⁰<https://tug.ctan.org/macros/latex/contrib/todonotes/todonotes.pdf>

file accordingly; I plan to eventually provide a class option to facilitate that without the need to edit the `.cls` file. If you feel like you want to help me with that, please let me know! You'll get gratitude and an acknowledgment here.

Chapter 2

Related Work

Several works have addressed in the analysis and identification of malware within the Node Package Manager (NPM) ecosystem, but none of them considered the evolution of the code with respect to all the published releases. One such tool is DONAPI [Hua+24a]. It uses a combination of static analysis, based on a predefined database of sensitive application programming interfaces (APIs), and dynamic analysis to extract behavioral patterns in order to detect and classify malicious packages.

An alternative to DONAPI is SpiderScan [Hua+24b], which is based on graph-based behavior modeling and matching. This tool uses LLM for various tasks, including the identification of sensitive APIs and the analysis of shell commands. Other tools also use LLM for NPM package analysis, such as the tool by Zahan et al. [Zah+25], Wang et al. [Wan+25], and Zhang et al. [Zha+25].

Although these tools achieve excellent results in identifying malicious packages, they are not open source and they also have limitations. The dynamic analysis in DONAPI, according to Huang et al. [Hua+24b], risks “hindering their practical usefulness in real-world scenarios“. Additionally, a common issue not specific to DONAPI and CEREBRO [Zha+25] is that most existing tools (including rule-based and learning-based methods) fail to holistically model malicious behavior. As noted by Huang et al. [Hua+24b], “these tools capture package behavior as a set of discrete features over sensitive APIs, code structures or semantics (e.g., data flow), simplifying the modeling of malicious behavior and thus hindering their practical effectiveness“. Furthermore, as discussed by Huang et al. [Hua+24b], “Prompt engineering methods leverage the capability of ChatGPT, and use self-refinement and chain-of-thought techniques to detect maliciousness, but can incur high expense in real-world scenarios“, besides the fact that in general LLM can be prone to hallucination, as he states Li et al. [Li+24].

In the work conducted by Weissberg et al. [Wei+25], it is argued that often simple tra-

ditional code metrics are enough to achieve results similar to, if not better than, those obtained with complex LLM.

This chapter is dedicated to the work that has been done by others in the field of your thesis. Don't write a mere list of papers with a summary of each. Instead, the goal is to give a picture of how your work is situated in the context of the work of others. In this case, you should think your skeptical reader is asking "hasn't this been solved already?".

There will be two main kinds of work that you will need to discuss: similar work to yours, and work that is complementary to yours.

Similar work will try to solve the same problem you are addressing, or a similar one. Here, focus on the differences: for example, you may write something along the lines of "Turing [Tur50] provides a naive definition of 'intelligent machine', which has this and that limitation. Our work is similar to that of Turing, but we address these problems by doing this and that."¹

Complementary work does not try to solve the same problem you are addressing, but it is important to get a full picture of your contribution. For example, if you are *not* attempting to solve a particular problem (it is "out of scope" for your work), but the solution to that problem is necessary for your work to be useful, you should explain that here.

The Related Work chapter may end up in other places of the document. Consider that it does two pieces of work at the same time: (i) substantiating the claim you made in the **Introduction** (Chapter 1) that your problem is relevant, and (ii) describe the context around your work to help understand its impact. If the latter part is more important, it may make sense to move this section to the end of the document, just before the **Conclusion** (Chapter 6). If the **Background** (Chapter 3) is not necessary to understand this chapter, it is also a good idea to move it just after the **Introduction** (Chapter 1).

2.1 Sources

While doing your review of related work, Google Scholar² is probably the easiest search engine for academic papers. You can be mindful of some bibliometric tricks to understand which papers are likely to be most influential in the field. You can see on Scholar the number of citations of a paper; CORE³ and Scimago⁴ provide rankings of conferences and

¹Do not be arrogant! I called the work of Turing naive as a joke. For sure your work will have limitations that other work may not have, and you should be honest about them.

²<https://scholar.google.com>

³<https://portal.core.edu.au/conf-ranks/>

⁴<https://www.scimagojr.com>

journals respectively. Non-peer-reviewed articles like those published only on arXiv⁵ may, of course, be important, but you should be more careful and skeptical about them. In case of doubt, ask for guidance from your advisor.

Another tool that you should *use with caution* is Semantic Scholar.⁶ It provides you with LLM-based tools to synthesize and ask questions about papers. Always double-check, because LLMs hallucinate and, as usual, you take responsibility for whatever you write in your document: treat LLMs as a useful tool you shouldn't trust.

Wikipedia is a great source of information, but you shouldn't consider it as reliable. Wikipedia articles should always provide the sources of the information they contain; you should check it yourself, judge its reliability, and cite the source.

In general, be careful about citing websites: make sure that they're authoritative, and when it's not obvious, justify why you consider them to be worth citing. Unlike this document, which I don't expect readers to print, your thesis must be readable in printed form: hence, make URLs readable in the text.

If a paper is very important for your own piece of work, have a look at the papers that cited it, to see if there are more recent works that are relevant to your thesis.

2.2 Style

Here are a few tricks about how to cite other pieces of work:

- If a paper is available in a peer-reviewed venue (journal or conference), cite that version rather than the non-reviewed one. Sometimes papers are published in two similar versions: one in a conference and an extended one in a journal. In that case, cite the journal.
- Use citations such as “The Hamiltonian cycle problem is NP-complete [Kar72]” or “Karp [Kar72] showed that the Hamiltonian cycle problem is NP-complete”. Do not write “[Kar72] showed that the Hamiltonian cycle problem is NP-complete”. Note, like in this case, that academics pretty much always refer to papers by citing authors' names rather than their titles.⁷
- DBLP⁸ is a good source of quality Bibtex records for papers you may want to cite.

⁵<https://arxiv.org>

⁶<https://www.semanticscholar.org/>

⁷If you use L^AT_EX, check the source code for how to cite paper authors using the `\textcite` command.

⁸<https://dblp.org>

- You may have a doubt about whether to put a link in the text or use a reference in the bibliography. Use a reference if it is a document you want to refer to (even if it's not a text document, like a video or audio), use a link if you just want to refer to something but do not use it as a reference document.
- If you decide to leave links, make it so that they're usable even in printed form. The `\myhref` command of this document can be an example of how to do it.

Chapter 3

Background

This chapter provides the necessary background information to understand the content of your thesis. The goal is to make your thesis *self-contained*: the reader, who you can assume will be a fellow Computer Science student, should be able to understand your work without having to refer to external sources. Of course, you should reference the sources you used to write this chapter, and references to documents that the reader can use to know more about those topics.

Do not include information that is not necessary to understand your work. If something is complementary to your work, but not useful to understand it, you can include it in the **Related Work** (Chapter 2).

Guide the reader while they are going through this chapter: do not just dump information, but present the context in which they will be used in the thesis, and what those concepts will be useful for.¹ Put forward references to the places these concepts are used, and put backward references from that chapter to here so that the reader can link the pieces.² Pay close attention, here and everywhere else in your thesis, to make sure that every non-trivial concept you use is defined clearly before being used.

Sometimes, there will be concepts that are tangential to your work: in those cases, avoid having an unclear or incomplete explanation: either explain it clearly in sufficient depth, or skip the concept altogether, leaving a bibliographic reference for those who want to understand the issue. A short but unclear explanation that will confuse readers is worse

¹A good exercise is thinking that your reader will constantly ask “so what?” after everything you tell them. If it is unclear why you provided them with information, you should explain (or, of course, remove the content if there is no explanation).

²Always keep in mind that the reader will not necessarily read all your documents. Such forward- and back-references help refresh the memory of those who read your documents linearly *and* help those who do not.

than either choice.

The Background chapter contains information about work that *you didn't do*. The work you did should be described in the **Methodology** (Chapter 4) instead.

The amount of information you will need to present in this chapter can vary a lot between theses. In some cases, you will have no or very little need for a Background chapter; in that case, it may make sense to move this information to a section in the Related Work chapter.

Chapter 4

Methodology

This chapter describes the core of your work: the methodology you used to solve the problem you described in the **Introduction** (Chapter 1). It is quite likely to have a different name (e.g., the name of the tool you wrote), and/or to be split into multiple chapters. *Methodology* suggests that you will spend space discussing your overall approach and justification for it: it is a good idea for a Master's Thesis, as opposed to just a list of things that you have done (which could be more correctly called *Method* or *Methods*, and would risk being boring and not very useful for the reader).

This chapter will likely be quite densely connected with things you have written in the **Background** (Chapter 3), and the results that you will show in the **Results** (Chapter 5). Put abundant references to help the reader navigate the document.

Try to keep a level of detail such that a skilled reader will be able to reproduce your result, but do not include unnecessary details that the reader can find out for themselves (e.g., the list of commands to perform a standard task). That is indeed valuable information that can help reproducibility, but if you do not think it is needed to understand your work, its place is likely to be in an appendix or in the documentation of the code you wrote, if you decide to release it.

This chapter should include work that has been done by you. If that is not the case, consider moving it to the **Background** (Chapter 3), or make it very clear that you are not the author. The same thing applies to images: if they aren't yours, you should cite the source.

The results of your work should generally not be included in this chapter. In some cases, preliminary results can be included to explain why you took a certain direction in your work (e.g., you took a choice rather than another because you measured that it was more efficient), but the overall evaluation of your work should be in the **Results** (Chapter 5).

You are likely to include algorithms in pseudocode. Check relevant L^AT_EX packages¹ for that.

¹<https://www.overleaf.com/learn/latex/Algorithms>

Chapter 5

Results

Here you will present the results of your work. Your skeptical reader is now asking themselves “is this working?”. You will show here to what extent it does. Be honest about the limitations: if your system doesn’t work in some cases, you should say so (and explain where and why, if you can). Like the [Methodology](#) chapter, this one may end up being split into multiple chapters.

Keep in mind that this shouldn’t be a mere dump of experimental results; you’re rather *teaching* your reader how and to what extent you managed to solve the problem you described in the [Introduction](#) (Chapter 1). If you have additional results that may be useful but are not necessary to understand the points you’re making (e.g., you evaluated your system on multiple datasets and the results all tell the same story), the place for them is in an appendix.

This chapter should have a lot of links to the methodology chapter(s), because you’re evaluating the choices you made there. If you developed a system made of multiple parts, make sure that you test them separately and together, so that the reader can understand how important each part is.

This chapter is likely to be quite full of figures and tables. Try to make them as informative as possible (e.g., use multiple lines in the same plot if possible). Showing graphs effectively is a complex art; try to spend some time on it and ask for guidance from your advisor. Whenever you have a figure or a table, make sure that you refer to it in the text (after all, if it’s not referred to in the text it means it has no part in the story you’re telling, so it has no place in this Chapter). Always use the text (and the caption, if you can) to explain what you want the reader to understand from the figure.

If you are using L^AT_EX, it will automatically place figures, tables, algorithms, etc. somewhere in the text. These parts of the documents are called *floats* because their position

Column 1	Column 2	Column 3
aaa	2	3.42
bbb	505	6.00
ccc	8	901.02

Table 5.1: A table using the `LATEX booktabs` package. Note there are no vertical rules. In case you want to do more fancy stuff (e.g., merging cells), check also the `multirow` and `multicol` packages. It’s generally a good idea for readability to align text to the left and numbers to the right (use the same number of significant digits).

floats around depending on `LATEX`’s will. There are options to advise `LATEX` about where floats should be placed. My advice is not to waste too much time on this and trust that they will end up in a good place. Generally, write the floats’ code before you refer to them: they will never be placed before the place they appear in.

For Figures, generally use vectorial formats (e.g., PDF, SVG) where it makes sense if you can. They will result in higher-quality images that are in most cases also smaller, leading to shorter compiling times and a smaller resulting PDF.

Tables look better without vertical rules: an example is in Table 5.1.

5.1 Discussion

At the end of this chapter, take a step back and summarize all the results so that the reader can understand the big picture. Sometimes, this becomes large and important enough to be worth a chapter on its own.

Chapter 6

Conclusion

You are finally done! Here you will summarize for the reader what you have taught them through the document, and the main takeaways of your work you would like them to remember.

It is also a good idea to discuss the limitations of your work and your views about what can be possible future work.

Bibliography

- [Hua+24a] Cheng Huang, Nannan Wang, Ziyang Wang, Siqi Sun, Lingzi Li, Junren Chen, Qianchong Zhao, Jiaxuan Han, Zhen Yang, and Lei Shi. *DONAPI: Malicious NPM Packages Detector using Behavior Sequence Knowledge Mapping*. 2024. arXiv: [2403.08334](https://arxiv.org/abs/2403.08334) [cs.CR]. URL: <https://arxiv.org/abs/2403.08334>.
- [Hua+24b] Yiheng Huang, Ruisi Wang, Wen Zheng, Zhuotong Zhou, Susheng Wu, Shulin Ke, Bihuan Chen, Shan Gao, and Xin Peng. “SpiderScan: Practical Detection of Malicious NPM Packages Based on Graph-Based Behavior Modeling and Matching”. In: *Proceedings of the 39th IEEE/ACM International Conference on Automated Software Engineering*. ASE ’24. Sacramento, CA, USA: Association for Computing Machinery, 2024, pp. 1146–1158. ISBN: 9798400712487. DOI: [10.1145/3691620.3695492](https://doi.org/10.1145/3691620.3695492). URL: <https://doi.org/10.1145/3691620.3695492>.
- [Kar72] Richard M. Karp. “Reducibility Among Combinatorial Problems”. In: *Proceedings of a symposium on the Complexity of Computer Computations, held March 20-22, 1972, at the IBM Thomas J. Watson Research Center, Yorktown Heights, New York, USA*. Ed. by Raymond E. Miller and James W. Thatcher. The IBM Research Symposia Series. Plenum Press, New York, 1972, pp. 85–103. DOI: [10.1007/978-1-4684-2001-2_9](https://doi.org/10.1007/978-1-4684-2001-2_9). URL: https://doi.org/10.1007/978-1-4684-2001-2_9.
- [Li+24] Siheng Li, Cheng Yang, Taiqiang Wu, Chufan Shi, Yuji Zhang, Xinyu Zhu, Zesen Cheng, Deng Cai, Mo Yu, Lemao Liu, Jie Zhou, Yujiu Yang, Ngai Wong, Xixin Wu, and Wai Lam. *A Survey on the Honesty of Large Language Models*. 2024. arXiv: [2409.18786](https://arxiv.org/abs/2409.18786) [cs.CL]. URL: <https://arxiv.org/abs/2409.18786>.
- [Tur50] Alan M. Turing. “Computing machinery and intelligence”. In: *Mind* LIX.236 (1950), pp. 433–460. DOI: [10.1093/MIND/LIX.236.433](https://doi.org/10.1093/MIND/LIX.236.433). URL: <https://doi.org/10.1093/mind/LIX.236.433>.
- [Wan+25] Jian Wang, Zhen Li, Jixiang Qu, Deqing Zou, Shouhuai Xu, Ziteng Xu, Zhenwei Wang, and Hai Jin. “MalPacDetector: An LLM-based Malicious NPM

- Package Detector”. In: *IEEE Transactions on Information Forensics and Security* PP (Jan. 2025), pp. 1–1. DOI: [10.1109/TIFS.2025.3580336](https://doi.org/10.1109/TIFS.2025.3580336).
- [Wei+25] Felix Weissberg, Lukas Pirch, Erik Imgrund, Jonas Möller, Thorsten Eisenhofer, and Konrad Rieck. “LLM-based Vulnerability Discovery through the Lens of Code Metrics”. In: *arXiv preprint arXiv:2509.19117* (2025).
- [Zah+25] Nusrat Zahan, Philipp Burckhardt, Mikola Lysenko, Feross Aboukhadijeh, and Laurie Williams. *Leveraging Large Language Models to Detect npm Malicious Packages*. 2025. arXiv: [2403.12196](https://arxiv.org/abs/2403.12196) [cs.CR]. URL: <https://arxiv.org/abs/2403.12196>.
- [Zha+25] Junan Zhang, Kaifeng Huang, Yiheng Huang, Bihuan Chen, Ruisi Wang, Chong Wang, and Xin Peng. “Killing Two Birds with One Stone: Malicious Package Detection in NPM and PyPI using a Single Model of Malicious Behavior Sequence”. In: *ACM Transactions on Software Engineering and Methodology* 34.4 (Apr. 2025), pp. 1–28. ISSN: 1557-7392. DOI: [10.1145/3705304](https://doi.org/10.1145/3705304). URL: <http://dx.doi.org/10.1145/3705304>.